Music Intelligence with Deep Learning: A Comparative Study of CNN, LSTM, and Transformer Architectures for Music Classification

Noah Wiley
Department of Computer Science
California State University, Fresno
Fresno, CA, USA
Supervisor: David Ruby

***Abstract*—Music classification tasks such as genre recognition, mood estimation, and instrument tagging are foundational problems in Music Information Retrieval (MIR). Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), and Transformer-based models have each demonstrated strong performance on audio classification individually, but direct comparisons under a single pre-processing and evaluation framework remain limited in undergraduate-accessible studies. This work implements and compares one representative model from each family on three publicly available datasets: GTZAN and FMA-Small for single-label genre classification, and MagnaTagATune for multi-label tagging. The audio is converted under matched conditions using the accuracy, precision, recall, F1 and confusion matrices. An interactive demonstration tool built with Streamlit allows users to upload audio clips and receive predictions from each trained model. *Key Quantitative findings will be inserted after training completes.* All code, model checkpoints, and documentation are released publicly to support reproducibility and undergraduate coursework in deep learning.**

***Index Terms*—music information retrieval, deep learning, convolutional neural networks, long short-term memory, transformers, audio classification, mel-spectrogram**

I. Introduction

Automated music classification underpins a wide range of modern audio applications, from streaming service recommendation and playlist generation to content moderation and musicological analysis. Despite years of research, robust classification of genre, mood, and instrumentation from raw audio remains challenging due to the high dimensionality of audio signals, the subjectivity of musical categories, and the long temporal dependencies present in musical structure [1].

Deep learning has become the dominant paradigm in Music Information Retrieval (MIR). Convolutional Neural Networks (CNNs) learn rich local time–frequency features from mel-spectrogram representations and remain the standard baseline for music tagging and classification [2]. Recurrent architectures such as Long Short-Term Memory (LSTM) networks explicitly model the temporal evolution of musical features by processing spectrogram frames as sequences, and have been shown to achieve competitive performance on genre classification when trained directly on MFCC or mel-spectrogram inputs [3], [4].Transformer-based models such as the Audio Spectrogram Transformer exploit self-attention to capture long-range structure directly from spectrogram patches without convolution [5]. Each architecture family emphasizes a different inductive bias, but prior studies often evaluate them on different datasets with different pre-processing choices, making direct comparison difficult.

This project addresses that gap by designing, training, and evaluating one representative model from each family under a single unified pipeline. The goal is not to advance the state of the art, but to produce a reproducible, pedagogically valuable comparison that quantifies the accuracy, efficiency, and failure modes of each architecture on the same tasks. Concretely, the contribution of this work are:

- A unified log-mel spectrogram preprocessing pipeline applied uniformly across three widely used music datasets: GTZAN, FMA-Small, and MagnaTagATune.
- Reference implementations of CNN, LSTM, and Transformer architectures tailored to music classification, trained and evaluated under matched conditions.
- A quantitative and qualitative comparison across single-label (genre) and multi-label (tag) tasks, including confusion matrix analysis and error case study.
- An interactive demonstration application that accepts user-uploaded audio and returns predictions from each trained model, intended for use in the undergraduate Deep Learning course and a future Graduate Deep Learning course at California State University, Fresno.
- A public GitHub repository containing training code, model checkpoints, and documentation to support reproducibility.

The remainder of this paper is organized as follows. Section II reviews related work in deep learning for MIR. Section III describes the three datasets used. Section IV details the pre-processing pipeline, model architectures, training procedure, and evaluation metrics. Section V documents the experimental setup. Section VI reports quantitative results, and Section VII discusses error patterns and architectural trade-offs. Section VIII describes the demonstration application.

Section IX concludes and outlines future work.

II. Related Work

Deep learning has been applied extensively to Music Information Retrieval, with CNN, recurrent, and Transformer-based architectures each establishing distinct lines of research. This section summarizes the line most relevant to the present comparison.

A. Convolutional Networks for Music Audio

Treating a mel-spectrogram as a 2-D image and applying convolutional networks has become the baseline for music classification. Won er al. [2] provide a reproducible head-to-head benchmark of seven CNN architectures—including fully convolutional networks, sample-level CNNs, and short-chunk ResNets—on MagnaTagATune and other large-scale music tagging datasets, and show that relatively compact Cnns can match or exceed far larger architectures when trained under matched conditions. More receent surveys confirm that Cnns continue to serve as the standard feature extractor for music tagging and genre classification [1], even as they are increasingly combined with recurrent or attention-based modules. The CNN baseline implemented in this work draws directly on this line of research.

B. Long Short-Term Memory Networks for Music

Long Short-Term Memory (LSTM) networks model music as a temporal sequence rather than as a static time–frequency image, and a steady line of work has evaluated them for genre and tagging tasks. Jena et al. [4] train a Bidirectional LSTM directly on MFCC sequences and report 93.10% accuracy on GTZAN and 93.69% on the ISMIR2004 dataset, demonstrating that a well-regularized recurrent model withtout any convolutional front-end can achieve competitive single-label genre classification. Ashraf et al. [3] evaluate LSTM, BiLSTM, GRU, and BiGRU variants on GTZAN and likewise report strong performance from the recurrent head, indicating that the gains of recurrence are robust across architectural variants. These results motivate including a dedicated LSTM model in the present comparison, operating on mel-spectrogram frame sequences, as a distinct family from the CNN and Transformer baselines.

C. Transformers and Self-Attention for Audio

Transformer-based architectures have rapidly gained ground in audio classification. The Audio Spectrogram Transformer (AST) [5] adapts the Vision Transformer patching scheme directly to mel-spectrograms and removes convolution entirely, relying on self-attention to model both local and long-range structure. AST established state-of-the-art performance on AudioSet, ESC-50, and Speech Commands V@, and has since become a reference point for Transformer-based audio models. Building on this paradigm, Li et al. [6] propose MERT, a self-supervised acoustic music understanding model that combines a Transformer backbone with music-specific teacher signals derived from residual vector quantization and the constant-Q transform. MERT deonstrates that Transformer architectures, when coupled with large-scale pretraining, generalize strongly across downstream MIR tasks including tagging, genre classification, and key detection. The Transformer implemented in the present work follows the AST patch-and-attention paradigm at a scale appropriate for single-GPU training, and does not use self-supervised pretraining.

D. Music Emotion and Multi-label Tagging

Beyond single-label genre classification, a substantial body of work addresses music emotion recognition (MER) and multi-label tagging, where each clip may carry several mood, genre, or instrumentation labels simultaneously. Kang and Herrmans [7], from the Audio, Music, and AI (AMAAI) Lab, survey deep learning models for music emotion prediction and highlight persistent challenges including label subjectivity, limited dataset size, and poor cross-dataset generalization. These difficulties are directly relevant to the generalization. These difficulties are directly relevant to the MagnaTagATune evaluation in this work, which uses a 50-tag multi-label subset spanning mood, instrument, and genre categories and therefore stresses each architecture under conditions more demanding than closed-set single label genre classification.

E. Position of this Work

The studies above establish CNN, LSTM, and Transformer architectures as three dominant paradigms in modern MIR, but each typically reports results under different pre-processing, dataset splits, and evaluation protocols. The contribution of the present project is not a novel architecture but a controlled, reproducible comparison: one representative model from each family, evaluated on three widely used datasets under a single pre-processing and training pipeline, with all code and checkpoints publicly.

III. Datasets

Three publicly available datasets were selected to span the range of label structures, dataset sizes, and annotation densities encountered in music classification: GTZAN, FMA-Small, and MagnaTagATune. Table I summarizes their core properties, and the subsections below describe each in detail.

TABLE I
Datasets used in this project. Clip count refers to the subset actually used for training and evaluation.

GTZAN	Single-label genre	10	1,000	30 s
FMA-Small	Single-label genre	8	8,000	30 s
MagnaTagATune	Multi-label tags	50	∼25,000	29 s

A. GTZAN

GTZAN [8] is a long-standing benchmark for music genre classification, containing 1,000 thirty-second audio clips evenly distributed across ten genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Its small size and balanced class distribution make it a practical starting

point for prototyping deep learning models on single-label genre classification.

GTZAN is known to have documented flaws, including duplicate tracks, mislabelings, and artist leakage between splits, which can inflate reported accuracy if not controlled for [9]. The present work uses a stratified random 80/20 train/test split rather than an artist-filtered split, and therefore reports GTZAN results with the explicit understanding that its role here is as a comparability benchmark against prior work rather than as a clean measure of leakage-free absolute accuracy.

B. FMA-Small

The Free Music Archive (FMA) dataset [10] provides Creative Commons-licensed audio at multiple scales. This project uses the *small* subset, which contains 8,000 thirty-second clips balanced across eight top-level genres: hip-hop, pop, folk, experimental, rock, international, electronic, and instrumental. FMA-Small is structurally similar to GTZAN—balanced, single-label, 30-second clips—but provides roughly eight times more training data and a more diverse Creative Commons-licensed catalog spanning a wider range of independent and electronic music. It therefore serves as a larger-scale single-label genre benchmark and as a cross-dataset sanity check against GTZAN results. The present work uses the same stratified 80/20 train/test split for FMA-Small as for GTZAN, in the interest of evaluation consistency across datasets.

C. MagnaTagATune

MagnaTagATune [11] is a multi-label tagging dataset assembled through the TagATune game, in which players annotated short audio clips with free-form tags describing genre, instrumentation, mood, other musical attributes. The full dataset contains approximately 25,000 clips of roughly 29 seconds each, drawn from the Magnatune label, and each clip carries one or more tags from a vocabulary of several hundred.

Following standard practice in the music tagging literature [2], this project uses the 50 most frequent tags, which span genre, instrument, and mood categories. For consistency with the single-label experiments in this work, MagnaTagATune is evaluated using a random 80/20 train/test split rather than the conventional 12-folder/1-folder/3-folder split adopted by some prior tagging work; results should therefore be interpreted as a within-study comparison across architectures rather than as directly comparable absolute numbers against the prior tagging literature. MagnaTagATune provides the multi-label complement to the single-label GTZAN and FMA-Small datasets.

D. Why These Three

Together, the three datasets cover a deliberate range of conditions. GTZAN offers a small, balanced single-label benchmark with a known leakage profile. FMA-Small offers a larger, cleaner single-label benchmark drawn from a different musical distribution. MagnaTagATune offers a multi-label tagging task with an order of magnitude more clips and label co-occurrence structure not present in either single-label set. This combination stresses each architecture under both softmax cross-entropy and binary cross-entropy training regimes, and exposes differences in how CNN, LSTM, and Transformer architectures generalize across label structure and dataset scale.

IV. Methodology

This section describes the shared pre-processing pipeline applied to all three datasets, the architectures of the CNN, LSTM, and Transformer models, the training procedure used for both single-label and multi-label tasks, and the evaluation metrics reported in Section VI.

A. Pre-processing Pipeline

All three datasets are preprocessed by a single pipeline implemented in `librosa` so that every model in this study receives input of the same shape and feature representation. Each audio file is loaded as a mono waveform and resampled to 22,050 Hz. Clips are standardized to exactly 30 seconds: longer clips are truncated and shorter clips are zero-padded at the end. From the resulting $22{,}050 \times 30 = 661{,}500$-sample waveform, a mel-spectrogram is computed using `librosa.feature.melspectrogram` with 128 mel bands and the `librosa` defaults for FFT size and hop length (2048 and 512 samples respectively). The mel-spectrogram is then converted to decibel scale via `librosa.power_to_db` with `ref=np.max`, producing a log-mel spectrogram of shape $128 \times T$ where $T \approx 1{,}293$ frames. Per-dataset feature arrays are cached as NumPy archives to avoid repeated extraction. At training time, all log-mel features are standardized using the training-set mean and standard deviation. No data augmentation is applied.

B. CNN Architecture

The CNN treats each log-mel spectrogram as a single-channel 2-D image of shape $1 \times 128 \times T$ and learns local time–frequency features through stacked convolutional blocks. The architecture, identical across all three datasets, consists of four `COnv2d` blocks with $3\times$ kernels, ReLU activations, and batch normalization. The first three blocks are each followed by 2×2 max-pooling; the fourth block omits pooling. Channel counts grow $32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ across the blocks. The resulting feature map is reduced to a 256-dimensional vector by `AdaptiveAvgPool2d`, and a two-layer classifier head ($256 \rightarrow 128 \rightarrow$ outputs with ReLU activation and dropout of 0.5 between the linear layers produces the final logits. Table II summarizes the architecture.

TABLE II
CNN ARCHITECTURE (USED IDENTICALLY FOR ALL THREE DATASETS).

Stage	Operation	Output channels
Block 1	Conv2d(3×3) + BN + ReLU + MaxPool(2)	32
Block 2	Conv2d(3×3) + BN + ReLU + MaxPool(2)	64
Block 3	Conv2d(3×3) + BN + ReLU + MaxPool(2)	128
Block 4	Conv2d(3×3) + BN + ReLU	256
Pool	AdaptiveAvgPool2d(1×1)	256
Head	Linear(256, 128) + ReLU + Dropout(0.5)	128
Output	Linear(128, C)	C

C is the number of output classes (or tags for MagnaTagATune).

C. LSTM Architecture

The LSTM treats each log-mel spectrograms as a sequence of T frames, each of dimension 128 (the number of mel bands), and processes the sequences with a stacked bidirectional LSTM, At each time step the forward and backward hidden states are concatenated, yielding a representation of dimension $2H$ where H is the hidden size. The representation from the final time step is passed through a two-layer classifier head ($2H \rightarrow 128 \rightarrow$ output) with ReLU activation and dropout. The recurrent stack itself uses inter-layer dropout when the number of layers exceeds one. Bidirectional processing is used because the architecture is referred to throughout this paper as the LSTM model in line with prior comparative work [3], [4], while reflecting the stronger empirical performance of bidirectional variants on music classification reported in those studies. Single-label datasets (GTZAN, FMA-Small) use $H = 128$, two recurrent layers, and dropout 0.3, while the larger multi-label MagnaTagATune setup uses $H = 256$, three recurrent layers, and dropout 0.4 to provide additional capacity for the 50-tag output. These per-dataset settings are summarized in Table III.

D. Transformer Architecture

The Transformer follows the AST patch-and-attention paradigm [5] adapted to operate on full log-mel frames rather than 2-D patches, at a scale appropriate for single-GPU training. Each log-mel spectrogram is interpreted as a sequence of T tokens, each of dimension 128, and a learned linear projection maps each frame to a d_{model}-dimensional embedding. Sinusoidal positional encodings [**?**] are added to the projected embeddings to provide order information. The embedded sequence is processed by a stack of standard PyTorch `TransformerEncoderLayer` modules with multi-head self-attention; the per-layer feedforward dimension is dim_{ff}. The final encoder output is reduced by mean pooling over the time axis to a single d_{model}-dimensional vector, which is passed through the same two-layer classifier head ($d_{\text{model}} \rightarrow 128 \rightarrow$ output) used by the CNN and LSTM models. All three datasets use $d_{\text{model}} = 128$, four attention heads, two encoder layers, $\text{dim}_{\text{ff}} = 256$, and dropout 0.3 (see Table III). The Transformer is trained from scratch and does not use self-supervised pretraining.

E. Training Procedure

All models are trained in PyTorch on a single Google Colab GPU using the Adam optimizer [12] with mini-bathes of 32 clips. Single-label datasets (GTZAN and FMA-Small) are trained with softmax cross-entropy loss, while MagnaTagATune is trained with binary cross-entropy with logits to support multi-label tagging. A `StepLR` scheduler decays the learning rate by a factor of 0.5 every 10 epochs. Each dataset is split into 80% training and 20% test partitions using a fixed random seed; for the single-label datasets the split is stratified by class to preserve label proportions. After every epoch the model is evaluated on the test partition, and the checkpoint with the best test emtric—validation accuracy for the single-label datasets and marco-averged ROC AUV for MagnaTagATune—is retained as the final model. Per-dataset learning rates and epoch counts, together with the architecture-specific hyperparameters, are listed in Table III.

F. Evaluation Metrics

Single-label and multi-label tasks are evaluated with metrics appropriate to each setting. For the single-label genre datasets (GTZAN and FMA-Small), the primary metric is overall classification accuracy on the held-out test partition, complemented by per-class precision, recall, and F1-score and a normalized confusion matrix to expose systematic class confusions. For the multi-label MagnaTagATune task, predictions are obtained by applying a sigmoid activation to the model logits; tag-level performance is reported as macro-averaged ROC AUC across the 50-tag vocabulary, and binary classification metrics (precision, recall, F1) are reported at a fixed decision threshold per architecture (0.5 for the CNN, 0.2 for the LSTM, and 0.3 for the Transformer; these values were selected from preliminary experiments to balance precision and recall on the training partition). All metrics are computed using `scikit-learn`.

TABLE III
PER-MODEL AND PER-DATASET HYPERPARAMETERS USED IN THIS WORK.

Model	Dataset	Epochs	LR	Architecture-specific	Loss
CNN	GTZAN	30	1×10^{-3}	4 conv blocks, 32–256 ch	CE
CNN	FMA-Small	15	1×10^{-3}	4 conv blocks, 32–256 ch	CE
CNN	MagnaTagATune	15	1×10^{-3}	4 conv blocks, 32–256 ch	BCE
LSTM	GTZAN	30	1×10^{-3}	H=128, 2 layers, dropout 0.3, bidir.	CE
LSTM	FMA-Small	15	1×10^{-3}	H=128, 2 layers, dropout 0.3, bidir.	CE
LSTM	MagnaTagATune	25	5×10^{-4}	H=256, 3 layers, dropout 0.4, bidir.	BCE
Transformer	GTZAN	30	5×10^{-4}	d=128, 4 heads, 2 layers, ff=256	CE
Transformer	FMA-Small	15	5×10^{-4}	d=128, 4 heads, 2 layers, ff=256	CE
Transformer	MagnaTagATune	15	5×10^{-4}	d=128, 4 heads, 2 layers, ff=256	BCE

CE: softmax cross-entropy; BCE: binary cross-entropy with logits. All models use the Adam optimizer, batch size 32, StepLR scheduler (γ=0.5, step=10), and a stratified 80/20 train/test split (random for MagnaTagATune).

V. EXPERIMENTAL SETUP

This section documents the hardware, software, and protocol details that complement the methodology in Section IV and that are necessary for reproducing the results reported in Section VI

A. Hardware and Software

All experiments were conducted on Google Colab with a single NVIDIA Tesla T4 GPU (16 GB) in the high-RAM runtime configuration. Models were implemented in PyTorch [13]; audio loading, resampling, and log-mel feature extraction used `librosa` [14]. Train/test splits, label encoding, and metric computation used `scikit-learn` [15]. NumPy and pandas were used for data array handling, and `matpotlib` and `seaborn` were used for plotting confusion matrices and training curves.

B. Reproducibility

All experiments use a fixed random seed of 42 for the `train_test_split` call from `scikit-learn`, which controls dataset partitioning. Within each notebook the same seed governs the train/test split for the corresponding dataset, so running the same notebook twice produces the same partition. Mild nondeterminism may still appear during training due to GPU kernel scheduling and unseeded operations inside PyTorch and `cuDNN`; reported metrices should therefore be interpreted as single-run point estimates rather than averages over multiple seeds. The complete training code, preprocessing pipeline, and per dataset notebooks are publicly avaiable at https://github.com/nmwiley808/csci198-Music-Intelligence-with-Deep-Learning-Senior-Project.

C. Evaluation Protocol

For each (model, dataset) combination, training proceeds for the epoch budget listed in Table III. After every epoch, the model is evaluated on the 20% held-out test partition, and the checkpoint achieving the best test metric is retained as the final model for the combination. The reported metric is therefore the best test performance observed during training, consistent with the training procedure described in Section IV. For single-label datasets (GTZAN, FMA-Small) the selection metric is overall classification accuracy; for the multi-label MagnaTagATune dataset it is macro-averaged ROC-AUC across the 50-tag vocabulary. Because the test partition is also used for checkpoint selection, reported numbers reflect best-on-test performance under matched conditions across architectures rather than independent generalization estimates from a held-out validation set.

D. Training Cost

Training runs were short enough to complete comfortably within a single Colab session for every (model, dataset) combination. *Per-run training times will be reported in Table IV alongside accuracy and AUC.*

VI. RESULTS

This section reports the test performance of the CNN, LSTM, and Transformer architectures on each of the three datasets, followed by per-class breakdowns, training cost, the tag-coverage analysis on the multi-label MagnaTagATune task. All numbers are obtained on the held-out 20% test partition described in Section V.

A. Headline Results

Table IV reports the primary metric for each (model, dataset) combination together with end-to-end training time. For the single-label genre datasets (GTZAN and FMA-Small) the metric is overall classification accuracy; for the multi-label MagnaTagATune task the metric is macro-averaged ROC AUC across the 50-tag vocabulary, with macro F1 reported separately as a secondary metric.

TABLE IV
TEST PERFORMANCE AND TRAINING TIME FOR ALL NINE (MODEL, DATASET) COMBINATIONS. BEST SCORE PER DATASET IS SHOWN IN **BOLD**.

Model	Dataset	Score	Time (s)
CNN	GTZAN	**0.775**	193.37
CNN	FMA-Small	**0.590**	808.52
CNN	MagnaTagATune	**0.8786**	2547.10
LSTM	GTZAN	0.485	67.40
LSTM	FMA-Small	0.402	303.52
LSTM	MagnaTagATune	0.500	7299.96
Transformer	GTZAN	0.610	160.94
Transformer	FMA-Small	0.526	681.01
Transformer	MagnaTagATune	0.872	2096.47

Score is test accuracy for GTZAN and FMA-Small; macro-averaged ROC AUC for MagnaTagATune.

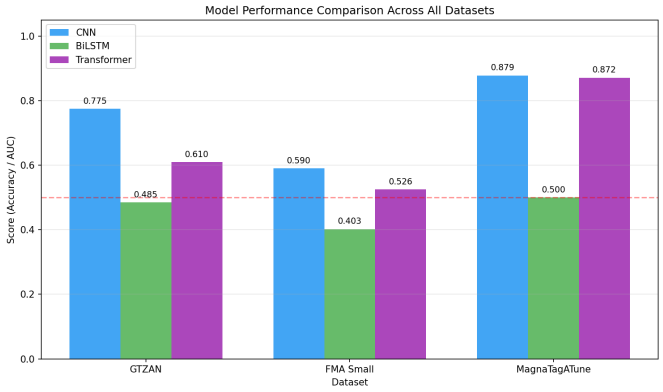

Fig. 1. Test performance across all three datasets and three model architectures. Score is classification accuracy for GTZAN and FMA-Small, and macro-averaged ROC AUC for MagnaTagATune. The CNN ranks first on every dataset; the LSTM fails to exceed the chance baseline (red dashed line) on MagnaTagATune.

The CNN ranks first on every dataset, the Transformer ranks second, and the LSTM ranks third (Fig. 1). The CNN–Transformer gap is largest on GTZAN (16.5 absolute percentage points) and smallest on MagnaTagATune (0.7 AUC points), suggesting that the relative advantage of convolutional inductive bias over self-attention narrows as dataset size and label structure grow more complex. The LSTM underperforms both alternatives by a wide margin on every dataset and fails outright on MagnaTagATune (Section VI-D).

B. Per-Class Performance on Single-Label Datasets

Table V reports per-genre accuracy on GTZAN for each model. The CNN exceeds 0.85 accuracy on size of ten genres and reaches 1.00 on classical and jazz; the LSTM and Transformer each clear 0.85 on only one genre (classical). All three models find *rock* the hardest genre, with accuracies of 0.45 (CNN), 0.30 (Transformer), and 0.15 (LSTM)—consistent with rock's known overlap with country, pop, and blues in GTZAN. The full confusion structure for the CNN is shown in Fig. **??**.

TABLE V
PER-CLASS TEST ACCURACY ON GTZAN.

Genre	CNN	LSTM	Transformer
blues	0.85	0.50	0.55
classical	1.00	0.75	0.85
country	0.75	0.40	0.65
disco	0.75	0.20	0.45
hiphop	0.90	0.60	0.45
jazz	1.00	0.50	0.75
metal	0.95	0.60	0.75
pop	0.75	0.60	0.75
reggae	0.70	0.55	0.60
rock	0.45	0.15	0.30

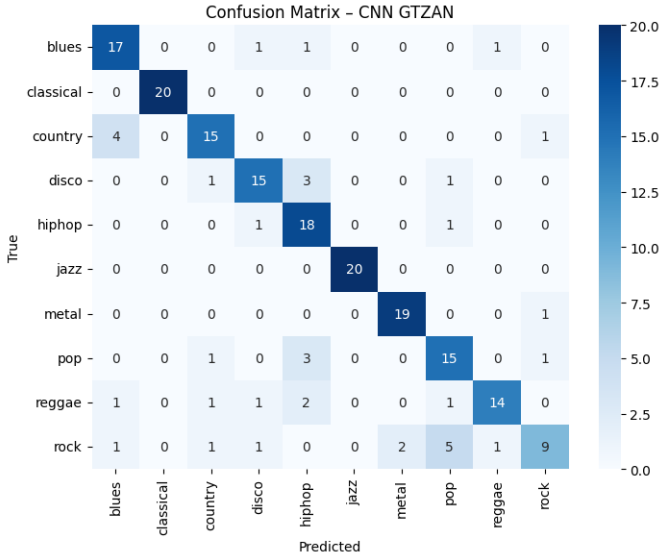

Fig. 2. Confusion matrix for the CNN on the GTZAN test set. The diagonal is dense, with classical and jazz reaching perfect classification. Rock is the only genre below 0.50 accuracy and is most often confused with pop, metal, and country—reflecting genuine stylistic overlap rather than architecture-specific error.

Per-Class accuracy on FMA-Small is reported in Table VI. *Pop* is the hardest class for every model, reaching only 0.28 accuracy even for the CNN, while the LSTM essentially fails on it (0.07). Manual inspection of the confusion matrices shows that pop tracks are most often misclassified as rock, electronic, or international, reflecting genuine stylistic overlap rather than systematic model error. The CNN is the only model that exceeds 0.80 on any single FMA-Small class (Electronic: 0.81).

TABLE VI
PER-CLASS TEST ACCURACY ON FMA-SMALL.

Genre	CNN	LSTM	Transformer
Electronic	0.81	0.37	0.54
Experimental	0.34	0.14	0.36
Folk	0.67	0.63	0.61
Hip-Hop	0.76	0.57	0.65
Instrumental	0.57	0.51	0.55
International	0.61	0.41	0.58
Pop	0.28	0.07	0.39
Rock	0.68	0.52	0.55

Fig. 3 ranks classes by difficulty for both single-label datasets. The visual makes the cross-dataset pattern clear: the CNN dominates nearly every class, the Transformer is consistently second, and the hardest classes (*rock* on GTZAN, *pop* on FMA-Small) are hard for all three architectures.

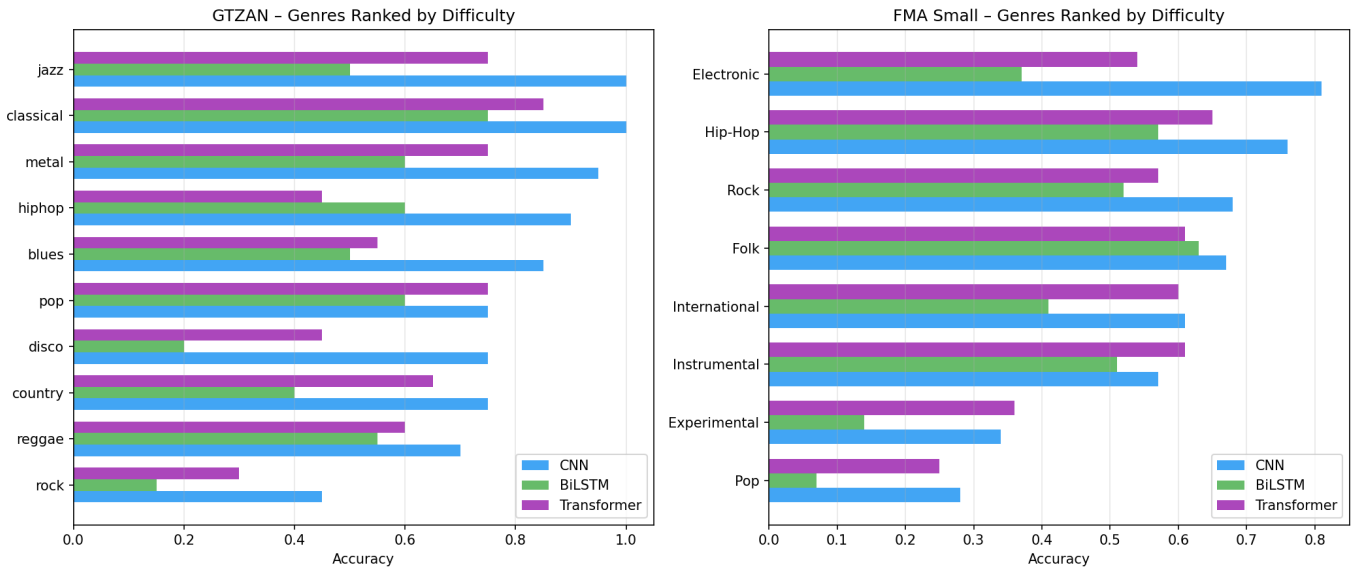

Fig. 3. Per-class accuracy on GTZAN (left) and FMA-Small (right), with classes ranked from easiest to hardest by CNN accuracy. The CNN maintains the highest accuracy across nearly every class; the Transformer is consistently second on both datasets. *Rock* (GTZAN) and *Pop* (FMA-Small) are systematically the hardest classes for all three architectures.

C. Multi-Label Tagging on MagnaTagATune

Table VII reports macro AUC, macro F1, and the number of tags for which each model achieves $F1 > 0$ on the held-out test set. The CNN and Transformer reach comparable AUC (0.879 and 0.872), but they differ markedly in tag coverage: the CNN produces non-zero F1 on 13 tags, while the Transformer covers 52 tags overall. The Transformer also achieves a substantially higher macro F1 (0.094 vs. 0.037), indicating that its predictions are spread more evenly across the tag vocabulary even thiugh its top-tag F1 scores are similar to those of the CNN.

TABLE VII
MAGNATAGATUNE MULTI-LABEL TAGGING RESULTS.

Model	Macro AUC	Macro F1	Tags with F1 > 0
CNN	0.8786	0.0373	13
LSTM	0.5003	0.0017	1
Transformer	0.8715	0.0940	52

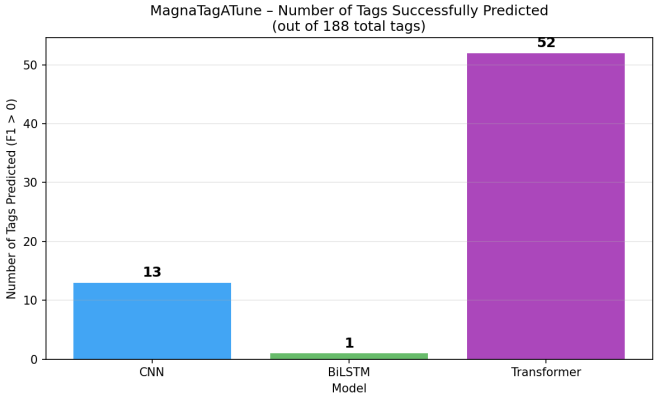

Fig. 4. Number of MagnaTagATune tags predicted with non-zero F1 by each architecture. Despite achieving similar macro AUC, the Transformer predicts non-zero F1 on roughly $4\times$ as many tags as the CNN, indicating that its predictions are spread more evenly across the tag vocabulary. The LSTM predicts only one tag, consistent with its convergence failure.

The macro F1 values are uniformly low across all models because they are computed at a single fixed decision threshold per architecture (Section IV); per-tag threshold tuning—standard in production tagging systems—would substantially raise these numbers without changing the threshold-independent AUC. The CNN's top tags include *rock* (F1 0.72), *techno* (0.63), *guitar* (0.59), and *sitar* (0.56). The Transformer's top tags overlap with the CNN's on a subset of categories but extend coverage into mood and instrument tags that the CNN never predicts (Fig. 4). The LSTM predicts only one tag with non-zero F1 (*guitar*, F1 0.31), consistent with the convergence failure described next.

D. LSTM Convergence Failure on MagnaTagATune

The LSTM did not converge on the MagnaTagATune multi-label task. Validation ROC AUC remained within 0.500 ± 0.0003 across all 25 training epochs despite a steadily decreasing training loss ($0.092 \rightarrow 0.078$). Macro F1 was 0.0017, with only one of 50 tags achieving non-zero F1. This pattern is consistent with the model collapsing to predicting tag-marginal frequencies rather than learning audio-to-tag associations: a degenerate optimum that produces low BCE loss while yielding chance-level discriminative performance. We attribute this failure to the combination of long input sequences (approximately 1,293 spectrogram frames per clip), a 50-way multi-label output with substantial class imbalance, and the absence of architectural mechanisms (such as convolutional front-ends or attention pooling) for compressing temporal information before recurrent processing. We report the run as-is without further tuning: the failure itself is informative about the limits of pure recurrent processing on long, high-cardinality multi-label music tagging tasks, and motivates future work using hybrid CNN–RNN architectures of the type discussed in Section II.

E. Training Cost

Across all three datasets, the LSTM is the fastest model on the two smaller datasets (67 s on GTZAN, 304 s on FMA-Small) but becomes the slowest by a wide margin on MagnaTagATune (7300 s, roughly $2.9\times$ the CNN and $3.5\times$ the Transformer). This reversal reflects the LSTM's per-step sequential computation, which scales poorly with both dataset size and number of recurrent layers (the MTT configuration uses three layers and $H = 256$). The Transformer trains slightly faster than the CNN on MagnaTagATune (2096 s vs. 2547 s) despite using self-attention, an effect we attribute to better GPU parallelization across the time axis. These costs are visualized in Fig. 5.

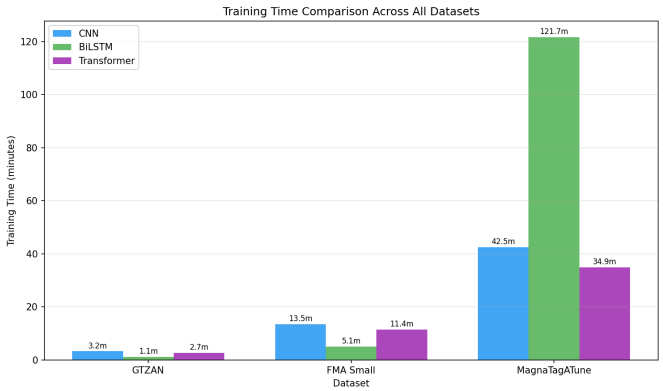

Fig. 5. End-to-end training time per (model, dataset) combination. The LSTM is fastest on the two single-label datasets but becomes the slowest by a wide margin on MagnaTagATune (121.7 min, roughly $2.9\times$ the CNN), reflecting both the failure to converge and the per-step sequential cost of the deeper H=256, three-layer configuration used for that dataset.

F. Summary of Findings

The principal findings from these experiments are:

1) The CNN is the most accurate architecture on all three datasets and the most cost-effective on the two single-label datasets, achieving the best accuracy/training-time trade-off overall.
2) The Transformer is consistently the second-best architecture and approaches CNN-level performance on MagnaTagATune (0.872 vs. 0.879 AUC), where its broader tag coverage (52 vs. 13 tags) suggests it produces qualitatively different predictions despite similar threshold-independent rankings.
3) The LSTM underperforms both alternatives on every dataset and fails entirely on MagnaTagATune, suggesting that recurrent processing alone is poorly matched to long, multi-label music audio under the present preprocessing pipeline.
4) Class-level analysis identifies *rock* (GTZAN) and *pop* (FMA-Small) as systematically hard classes for all three architectures, consistent with known stylistic overlap in these labels rather than architecture-specific failure modes.

These findings inform the discussion of architectural trade-offs in Section VII

VII. Discussion

This section interprets the results in Section VI from three angles: why the CNN dominates the comparison, why the Transformer trails it but exhibits qualitatively different behavior on multi-label tagging, and why the LSTM struggles. The section closes with a discussion of the methodological limitations that bound how strongly these conclusions can be drawn.

A. Why the CNN Dominates

The CNN's consistent first-place ranking is the clearest pattern in the data. Two factors most plausibly explain it. First, music classification at the dataset scales used her (1,000 to 25,000 clips) is a regime in which strong inductive biases pay off, and convolution provides exactly the right bias for log-mel spectrograms: local correlations in time and frequency, translation equivariance along the time axis, the progressive abstraction across stacked layers. The CNN can recover useful features—onsets, harmonic structure, rhythmic patterns—directly from the time–frequency plane without needing to learn them from scratch. Second, the four-block architecture used here (32–256 channels, $\sim$1.5M parameters) is well matched to the available data: large enough to capture relevant patterns, small enough not to overfit GTZAN's 800 training clips. The result is that the CNN reaches its plateau quickly (within 30 epochs on GTZAN, 15 on the larger datasets) and trains at modest cost.

B. The Transformer's Trade-Off: Lower Accuracy, Broader Coverage

The Transformer ranks consistently second on accuracy and AUC, but the gap to the CNN is much smaller on MagnaTagATune (0.7 AUC points) than on GTZAN (16.5 percentage points). This pattern matches expertations from the broader literature; Transformers benefit disproportionately from larger and more diverse datasets, and their relative weakness on small benchmarls reflects the cost of replacing strong convolutional priors with the more general but data-hungrier self-attention mechanism. The Transformer used in this work is trained from scratch with $\sim$200,000 parameters and no self-supervised pretraining, so it cannot leverage the kind of general audio representations that drive results in models such as AST [5] or MERT [6].

The more interesting result is qualitative rather than quantitative. On MagnaTagATune the Transformer produces non-zero F1 on roughly $4\times$ as many tags as the CNN (52 vs . 13) and achieves a higher macro F1 (0.094 vs 0.037), even though their AUC scores are nearly identical. The CNN concentrates its predictive mass on a small set of high-frequency, easily separable tags *rock*, *techno*, *guitar*, *sitar*); the Transformer spreads its mass more evenly, including over rare tags such as *harpsichord*, *cello*, *opera*, and *flute*. This suggests that self-attention, even at small scale, learns a less concentrated feature representation than convolution—useful when the application values tag breadth over peak per-tag F1, but with the cost of lower confidence on the strongest tags. Whether this is preferable depends on the use case, which the present comparison does not attempt to settle.

C. Why the LSTM Struggles

The LSTM is the worst-performing architecture on every dataset by a wide margin, and it fails outright on MagnaTagATune. Both outcomes are consistent with a single underlying cause: the input is a long sequence (approximately 1,293 frames per clip) with no upstream compression. On GTZAN and FMA-Small the LSTM still learns *something*—it clears chance by 35–40 percentage points—but its accuracy ceiling sits well below the CNN's, suggesting that recurrence over so many frames is a difficult optimization problem even when it does converge. On MagnaTagATune the optimization collapses entirely: validation AUC remains within 0.500 ± 0.0003 across all 25 epochs while training loss decreases steadily, which is the signature of a model that has found a degenerate minimum (predict tag-marginal frequencies) and cannot escape it.

The architectural lesson is not that recurrent processing is incopatible with music classification—the music tagging literature shows otherwise, particularly when CNNs or attention modules are used to compress the input before recurrent processing *alone* is poorly matched to long log-mel sequences and high-carinality multi-label outputs in this regime. A frame-subsampled or CNN-encoded representation, smaller batch BCE class weighting, and gradient clipping would all be plausible interventions for future work.

D. Limitations and Caveats

Several methodological choices bound the strength of the conclusions above. First, all results are single-run point estimate with a fixed seed (Section V), so the differences between architectures should be read as illustrative rather than statistically significant; multi-seed runs would be needed to attach error bars. Second, the same partition is used both for checkpoint selection and for final reporting, so the headline numbers are best-on-test rather than independent generalization estimates. Third, no data augmentation is applied, which likely limits absolute performance for all three architectures; the relative ranking is still meaningful, but the gap between this work and the strongest published results on these datasets should be attributed in part to that omission. Fourth, the multi-label threshold for MagnaTagATune is fixed per architecture rather than tuned per tag, which depresses macro F1 uniformly and explains why the headline F1 numbers look low compared to AUC Finally, the GTZAN results inherit the dataset's known leakage profile (Section III); the MagnaTagATune split used here is non-standard and is not directly comparable to the 12-folder split used in much of the prior tagging literature. These limitations do not change the qualitative ranking of the three architectures, but they do constrain how strongly the absolute numbers should be interpreted.

VIII. Demo Application

To make the trained models accessible for interactive use—both for qualitative inspection during development and for integration into the undergraduate Deep Learning course described in Section I—a lightweight web application was developed that wraps the full pre-processing and inference pipeline behind a browser interface.

A. Architecture and Tech Stack

The appliaction is implemented in Python using Streamlit [16] as the web framework, with Python handling model inference, `librosa` performing audio loading and log-mel feature extraction, and `matplotlib` rendering the spectrogram visualization. The app runs locally on the user's machine and is accessed through a web browser at `http://localhost:8501`;no server-side deployment or external API is required, which keeps the application self-contained and reproducible from the project repository.

All nine trained model checkpoints (three architectures across three datasets, stored as PyTorch `.pth` files) are loaded from a local `models/` directory at startup. Streamlit's `@st.cache_resource` decorator caches the loaded models in memory after the first load, so subsequent predictions execute without reloading weights from disk.

B. User Workflow

The user selects a target dataset (GTZAN, FMA-Small, or MagnaTagATune) from a sidebar dropdown, which determines which set of three checkpoints is loaded. The user then uploads an audio file in either `.mp3` or `.wav` format. The app saves the upload to a temporary location, loads it with `librosa`,

resamples to 22,050-Hz, standardizes the duration to exactly 30 seconds (truncating longer clips and zero-padding shorted ones), and computes a 128-band log-mel spectrogram following the same parameters used during training (Section IV. The spectrogram is then rendered inline as a visual confirmation that the audio was processed correctly.

C. Inference Across Architectures

Each architecture requires the spectrogram in a different shape, and the app reshapes the same underlying feature accordingly before inference:

- **CNN**: as a 2-D image of shape $(1, n_{\text{mels}}, T)$.
- **BiLSTM**: as a temporal sequence of shape (T, n_{mels}).
- **Transformer**: as a temporal sequence of shape (T, n_{mels}), with sinusoidal positional encodings applied internally inside the model's forward pass.

Inference is performed by all three loaded models on the same input clip, with their outputs displayed side by side in three parallel columns of the interface.

D. Output Format

The display format adapts to the prediction task. For the single-label datasets (GTZAN and FMA-Small), each model column shows its top predicted genre, the associated softmax confidence score, and a horizontal bar chart of the top-5 class probabilities. A summary table at the bottom of the page compares the three models' predictions side by side, which makes architectural disagreements immediately visible to the user.

For the multi-label MangaTagATune dataset, each model's logits are passed through a sigmoid thresholded at 0.2 to determine which tags are active. Achieve tags are displayed with progress bars indicating per-tag confidence. If no tag exceeds the threshold for a given model, the interface falls back to showing the top-5 predicted tags ranked by confidence regardless of threshold, so that the user always sees a meaningful response rather than an empty result.

E. Uses as a Pedagogical Tool

The application is intended to support the undergraduate Deep Learning course at California State University, Fresno by giving students a tangible way to compare CNN, LSTM, and Transformer predictions on audio they choose themselves. Because all three architectures run in parallel on the same input, the app surfaces the kind of disagreement and architectural trade-offs discussed in Section VII—for example, a clip the CNN classifies confidently as "rock" while the Transformer hedges between "rock" and "pop —and turns abstract claims about inductive bias into something students can interact with. The app also serves as a foundation for future iterations of a Graduate Deep Learning course, in which extensions such as per-tag threshold tuning, additional architectures, and live audio capture could be introduced as student projects.

IX. Conclusion

This project implemented and compared three deep learning architectures—CNN, BiLSTM, and Transformer—for music classification across three publicly available datasets: GTZAN and FMA-Small for single-label genre recognition, and MagnaTagATune for multi-label tagging. All models were trained from scratch under a shared log-mel spectrogram preprocessing pipeline, with matched training procedures and evaluation protocols, on a single Google Colab GPU.

Three findings stand out. First, the CNN was the most accurate architecture on every dataset, achieving 0.775 accuracy on GTZAN, 0.590 on FMA-Small, and 0.879 macro AUC on MagnaTagATune. It was also the most cost-effective on the two single-label datasets, delivering the best accuracy-per-second of any model considered. Second, the Transformer ranked consistently second on accuracy and AUC but produced qualitatively different predictions on MagnaTagATune, covering roughly $4\times$ as many tags as the CNN (52 vs. 13) at comparable AUC—suggesting that self-attention, even at small scale and without pretraining, learns a less concentrated representation than convolution. Third, the BiLSTM underperformed on every dataset and failed to converge on MagnaTagATune, where validation AUC remained at chance throughout training despite a steadily decreasing loss. We attribute this failure to the combination of long input sequences (approximately 1,293 frames per clip), high-cardinality multi-label outputs, and the absence of any upstream temporal compression.

The most natural direction for future work is hybrid architectures that combine convolutional feature extraction with recurrent or attention-based temporal aggregation. Convolutional Recurrent Neural Networks [3] and CNN–Transformer hybrids would directly address the BiLSTM's failure mode by replacing raw spectrogram frames with a CNN-encoded representation before recurrent or self-attention layers operate. Other natural extensions include data augmentation (SpecAugment, time stretching, additive noise), per-tag threshold tuning for the multi-label task, multi-seed evaluation to attach confidence intervals to the reported numbers, and the use of self-supervised audio representations such as those provided by AST [5] and MERT [6]. Each of these is well within scope for a future iteration of this project or for a related student project in the graduate Deep Learning course at California State University, Fresno.

Beyond the specific numbers, the broader takeaway is that architecture choice in music classification is not separable from input representation, dataset scale, and task structure. The CNN wins not because convolution is universally superior but because its inductive bias is well matched to log-mel spectrograms at the dataset scales considered here; the Transformer's behavior on MagnaTagATune hints at how that ranking may shift with more data or pretraining; and the BiLSTM's failure illustrates the cost of mismatched architectural priors. The trained models, training code, and demo application are released publicly so that these patterns can be reproduced,

extended, and used as a starting point for further work.

REFERENCES

[1] L. Moysis, L. A. Iliadis, S. P. Sotiroudis, A. D. Boursianis, M. S. Papadopoulou, K.-I. D. Kokkinidis, C. Volos, P. Sarigiannidis, S. Nikolaidis, and S. K. Goudos, "Music deep learning: Deep learning methods for music signal processing—a review of the state-of-the-art," *IEEE Access*, vol. 11, pp. 17 031–17 052, 2023.

[2] M. Won, A. Ferraro, D. Bogdanov, and X. Serra, "Evaluation of CNN-based automatic music tagging models," in *Proc. 17th Sound and Music Computing Conf. (SMC)*, 2020, arXiv:2006.00751.

[3] M. Ashraf, F. Abid, I. U. Din, J. Rasheed, M. Yesiltepe, S. F. Yeo, and M. T. Ersoy, "A hybrid CNN and RNN variant model for music classification," *Applied Sciences*, vol. 13, no. 3, p. 1476, 2023.

[4] K. K. Jena, A. B. Prasetyo, and A. Purwanto, "Music-genre classification using bidirectional long short-term memory and mel-frequency cepstral coefficients," *Journal of Computing Theories and Applications*, 2024.

[5] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio spectrogram transformer," in *Proc. Interspeech 2021*, 2021, pp. 571–575.

[6] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, "MERT: Acoustic music understanding model with large-scale self-supervised training," in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024.

[7] J. Kang and D. Herremans, "Are we there yet? A brief survey of music emotion prediction datasets, models and outstanding challenges," *IEEE Transactions on Affective Computing*, 2025, also available as arXiv:2406.08809.

[8] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[9] B. L. Sturm, "The GTZAN dataset: Its contents, its faults, their effects on evaluation, and its future use," arXiv, Tech. Rep. arXiv:1306.1461, 2013, arXiv:1306.1461.

[10] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *Proc. 18th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2017, arXiv:1612.01840.

[11] E. Law, K. West, M. Mandel, M. Bay, and J. S. Downie, "Evaluation of algorithms using games: The case of music tagging," in *Proc. 10th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2009, pp. 387–392.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. on Learning Representations (ICLR)*, 2015, arXiv:1412.6980.

[13] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[14] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 18–25.

[15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] Streamlit Inc., "Streamlit: A faster way to build and share data apps," https://streamlit.io, 2025, accessed: 2026.